学　　号　：

202209547

上海立达学院

SHANGHAI LIDA UNIVERSITY

本科毕业论文（设计）

基于 Web 的大学生志愿者管理系统设计与开发

学院名称　：数字科学学院

专业名称　：数据科学与大数据技术

学生姓名　：王冠超

指导教师　：朱媛媛

2026　年　3　月　20　日

BACHELOR'S DEGREE THESIS OF SHANGHAI LIDA UNIVERSITY

Design and Development of a Web-Based College Student Volunteer Management System

Candidate：Wang Guanchao

Supervisor：Zhu Yuanyuan

Mar. 20th, 2026

郑重声明

本人郑重声明：所呈交的论文是本人在导师的指导下独立进行研究所取得的研究成果。除了文中特别加以标注引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写的成果作品。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。本人完全意识到本声明的法律后果由本人承担。

毕业论文（设计）成果归上海立达学院所有。

作者专业　：　数据科学与大数据技术

作者学号　：　202209547

作者签名　：

年　月　日

基于 Web 的大学生志愿者管理系统设计与开发

摘　要

随着高校志愿服务活动日益常态化、制度化，传统依赖 Excel、纸质表单和人工统计的管理方式已经难以满足活动发布、报名审核、签到签退、服务时长认定和统计分析等业务要求。本文结合上海立达学院大学生志愿服务管理场景，设计并实现了一套基于 Web 的大学生志愿者管理系统。系统采用前后端分离的 B/S 架构，前端使用 Vue 3、Element Plus、Pinia 与 ECharts 构建交互界面，后端基于 Spring Boot、MyBatis-Plus、JWT 与 H2/MySQL 完成业务处理与数据持久化。围绕学生端与管理员端双角色需求，系统实现了活动发布、报名审核、签到签退、服务时长生成与确认、公告消息推送、统计报表展示与数据导出等核心功能。本文在系统分析的基础上完成了业务流程设计、数据库建模、模块划分和关键代码实现，并结合实际运行截图、测试用例和统计结果对系统可用性进行验证。研究结果表明，该系统能够较好支撑高校志愿服务管理的信息化需求，具有部署简单、功能闭环完整、界面清晰和易于扩展等特点，可为高校志愿服务数字化管理提供一定参考。

关键词：Web 系统；志愿者管理；Spring Boot；Vue 3；高校信息化

BACHELOR'S DEGREE THESIS OF SHANGHAI LIDA UNIVERSITY

ABSTRACT

Design and Development of a Web-Based College Student Volunteer Management System

With volunteer activities in colleges becoming increasingly normalized and standardized, the traditional management mode based on spreadsheets, paper forms, and manual statistics can no longer satisfy the business requirements of activity publishing, signup review, check-in and check-out, service-hour confirmation, and statistical analysis. Focusing on the volunteer-service scenario of Shanghai Lida University, this thesis designs and implements a web-based college student volunteer management system. The system adopts a browser/server architecture with front-end and back-end separation. Vue 3, Element Plus, Pinia, and ECharts are used on the front end, while Spring Boot, MyBatis-Plus, JWT, and H2/MySQL are adopted on the back end for business processing and data persistence. Oriented to both student and administrator roles, the system supports activity publishing, signup review, check-in and check-out, service-hour generation and confirmation, notice delivery, statistical reporting, and data export. Based on project source code, this thesis describes business-process design, database modeling, module decomposition, and key-code implementation, and verifies the practicality of the system through screenshots, test cases, and runtime results. The study shows that the system can effectively support the information management needs of college volunteer services and provides a feasible reference for digital campus volunteer-service platforms.

Key words: Web system; volunteer management; Spring Boot; Vue 3; campus informatization

目 录

1 绪论 1
1.1 研究背景与意义 1
1.2 国内外发展现状 1
1.3 研究内容与技术路线 2
2 相关技术与理论基础 3
2.1 B/S 架构与 REST 风格接口 3
2.2 后端技术体系 3
2.3 前端技术体系 4
2.4 数据持久化与运行环境 4
3 系统需求分析与数据库设计 6
3.1 角色需求与业务流程分析 6
3.2 功能性与非功能性需求 7
3.3 数据需求分析与概念模型 8
3.4 逻辑结构设计 8
3.4.1 账户与基础档案表设计 9
3.4.2 活动主数据与通知表设计 12
3.4.3 活动过程数据表设计 14
4 系统架构与功能模块设计 18
4.1 系统总体架构设计 18
4.2 功能模块设计 18
4.3 关键业务机制设计 21
4.4 前后端协同与交互设计 22
5 系统实现与关键技术分析 24
5.1 认证授权实现 24
5.2 活动管理与报名审核实现 25
5.3 学生报名、签到签退与时长生成实现 26
5.4 公告推送与统计报表实现 29
6 系统测试与结果分析 31
6.1 测试环境与测试范围 31
6.2 功能测试结果 31
6.3 结果分析 32

6.4 本章小结 33
7 结论与展望 34
7.1 研究结论 34
7.2 后续展望 34
参考文献 35
致　谢 36

1 绪论

1.1 研究背景与意义

高校志愿服务既是大学生社会责任教育的重要载体，也是校园治理与社会服务协同的重要组成部分。随着新时代志愿服务体系建设不断深化，志愿服务工作正在从单次组织、分散登记逐步走向制度化、常态化和数字化管理[13][14]。对于高校而言，志愿活动不仅涉及活动发布和人员组织，更关系到志愿时长认定、活动质量评价、学生激励机制以及德育数据留痕等一系列管理事项。若仍然采用人工登记、纸质签到和线下汇总的方式，不仅效率低，而且容易出现信息遗漏、重复登记、统计口径不统一等问题。

从项目实践看，大学生志愿活动具有组织频次高、参与人员多、时间窗口短、过程记录要求严格等特征。一场完整的活动通常包含活动创建、报名、审核、签到、签退、时长确认、公告通知和报表分析等多个环节，每个环节都需要保证数据一致性与可追溯性。传统方式将这些环节割裂处理，导致活动组织者很难快速掌握报名进度、到场情况和累计服务时长，也不利于学生及时查询个人记录。基于 Web 的管理系统能够依托浏览器统一承载功能入口，兼顾跨平台访问、集中维护和快速部署，因此成为校园管理信息化建设中的合理技术路径。

本课题选择“基于 Web 的大学生志愿者管理系统设计与开发”作为研究对象，具有较强的现实针对性和实践价值。一方面，系统将高校志愿服务的核心业务流程固化为可执行的数字规则，有助于提升活动管理效率、降低人工错误率、增强数据留痕能力；另一方面，通过完成系统设计、数据库建模、前后端开发和测试验证，也能够较完整地体现软件工程专业训练成果，为学生综合运用 Java Web 开发、前端工程化、数据库设计与文档写作能力提供实践载体。

1.2 国内外发展现状

从 Web 系统架构演进角度看，Fielding 在 REST 架构研究中提出了基于资源、统一接口与分层约束的网络软件设计思想，为现代 Web 应用的接口组织方式提供了理论基础[1]。当前面向管理业务的信息系统普遍采用浏览器/服务器模式，利用标准化 HTTP 接口完成前后端协同，再结合关系数据库与身份认证机制支撑业务闭环。随着 Java 生态和前端工程化体系不断成熟，基于 Spring Boot、Vue 等框架构建校园管理系统已成为较为常见的实现方式[3][4]。

国外高校与公益组织在志愿者信息管理方面起步较早，其重点通常放在流程协同、在线服务、自助报名、数据归档与多角色权限控制上。这类系统强调通过

统一平台提升活动透明度和组织效率，并在统计分析中关注参与频率、服务时长和资源匹配效果。国内高校在志愿服务管理数字化方面近年发展较快，但很多系统仍依附于学生工作平台或独立小程序，存在模块碎片化、统计口径不统一、活动全周期管理不足等问题。尤其在报名审核、签到签退、时长认定和消息提醒联动方面，往往缺乏真正闭环的系统支持。

从技术发展趋势看，轻量化开发框架、组件化前端和数据可视化工具的成熟，为构建可维护、可扩展的校园业务系统提供了良好条件。Spring Boot 通过自动配置、统一依赖管理和分层组织降低了后端开发成本[3]；Vue 3、Vue Router、Pinia 与 Element Plus 则以响应式机制、路由组织和组件化设计提升了前端开发效率[4][5][6]；ECharts 等可视化库使管理者能够直观掌握活动数量、报名情况和服务时长分布[7]。因此，本课题在技术选型上具备较好的现实基础。

1.3 研究内容与技术路线

本文围绕大学生志愿服务管理的业务实际，结合已完成的项目源码，对系统需求、架构设计、数据库设计、功能实现和测试验证进行系统论述。全文首先分析项目建设背景、应用意义与相关技术基础，然后从角色需求、业务流程和数据实体三个层面完成需求分析与数据库建模；在此基础上给出系统总体架构与功能模块设计方案，进一步结合关键源码说明认证授权、活动管理、签到签退、服务时长生成和统计报表等核心功能的实现过程，最后通过测试结果评估系统的可用性与局限性。

在研究方法上，本文采用文献分析法、软件工程分析法与系统实现验证相结合的方式推进。文献分析法用于明确 Web 架构、身份认证与校园信息系统建设的相关技术背景；软件工程分析法用于拆解角色、功能、流程、数据和接口；系统实现验证则基于项目源码、数据库脚本、界面截图与测试结果，对论文中的设计结论进行实证支撑。通过上述路径，本文力求做到“论文叙述有源码依据、功能分析有界面支撑、技术论证有可信文献来源”。

从论文目标来看，本文并非停留在一般性的页面功能说明，而是试图回答三个问题：其一，高校志愿服务管理的关键业务闭环应如何被系统化表达；其二，在毕业设计边界内，如何选择一条既保证实现可行、又具备较强展示价值的技术路线；其三，如何通过数据库结构、核心代码和测试证据证明系统设计的合理性。围绕这三个问题展开分析，有助于使论文内容从“做了什么”进一步提升到“为什么这样做、这样做是否有效”的层面。

2 相关技术与理论基础

2.1 B/S 架构与 REST 风格接口

本系统采用典型的 B/S 架构，即以浏览器作为客户端入口、以后端服务提供统一业务接口和数据访问能力。该模式不要求终端安装专用客户端，学生与管理员只需通过浏览器即可完成登录、活动处理和报表查询，适合高校内部应用快速部署与统一维护的场景。相较于传统 C/S 架构，B/S 模式在运维成本、版本一致性和跨平台访问方面具有明显优势。

在接口设计上，系统遵循资源导向和统一接口的组织思路，将认证、学生端业务和管理端业务划分为不同的接口集合。例如登录接口负责用户令牌签发，学生端接口围绕活动、报名、签到和消息展开，管理端接口则聚焦学生管理、活动发布、审核处理和报表统计。该设计方式与 REST 架构关于资源表达、分层处理与可扩展交互的思想一致[1]，有利于前后端协同开发，也便于后续扩展移动端或第三方集成能力。

2.2 后端技术体系

后端以 Java 17 作为运行基础。JDK 17 作为长期支持版本，提供了稳定的语言特性与标准类库支持，能够满足当前项目在集合处理、日期时间、加解密与并发基础设施方面的需求[2]。在项目构建层面，系统通过 Maven 管理依赖与生命周期，后端核心框架为 Spring Boot 3.4.4。Spring Boot 提供自动配置、内嵌容器和统一配置体系，使系统能够快速完成 Web 服务、参数校验、测试环境等基础能力建设[3]。

数据访问层使用 MyBatis-Plus 3.5.10.1。相较于直接编写大量基础 SQL 映射代码，MyBatis-Plus 在保持 MyBatis 灵活性的同时，提供了通用 Mapper、条件构造器和逻辑删除配置等能力，可有效降低常规增删改查代码量[8]。从源码可见，项目在学生、活动、报名、签到、时长等实体的查询过程中广泛使用 LambdaQueryWrapper 与 LambdaUpdateWrapper，以保证条件表达清晰、类型安全且便于维护。

在认证机制方面，系统使用 JJWT 实现 JSON Web Token 的生成与解析。JWT 通过在令牌中封装用户标识、角色代码和关联档案标识，使服务端能够在无状态条件下快速恢复当前登录用户身份[9]。本系统在登录成功后由后端签发包含 userId、username、roleCode 和 refId 的令牌，并由前端在后续请求中统一携带，

从而实现学生和管理员两类角色的接口访问控制。

需要说明的是，教学型项目与生产型项目在安全强度上存在差异。本系统密码摘要采用 SHA-256 方案，能够避免明文存储，但尚未进一步引入强口令哈希算法、盐值策略和多因素认证机制，因此更适合作为校园教学项目或小规模内部演示系统。在后续扩展中，可进一步结合 BCrypt、审计告警和权限细化策略提升系统安全水平。

2.3 前端技术体系

前端部分采用 Vue 3 作为核心框架。Vue 3 的响应式系统和组合式开发思想有利于组织表单、表格、图表与页面状态，使界面逻辑与业务逻辑能够较清晰地分层表达[4]。路由层通过 Vue Router 管理登录页、学生端页面和管理端页面之间的导航切换[5]；状态管理则借助 Pinia 实现用户信息、令牌和公共状态的统一维护，减少页面间重复获取数据的复杂度。

界面组件方面，系统选用了 Element Plus。该组件库提供表格、表单、对话框、菜单、卡片和消息提示等丰富组件，适合后台信息管理系统的快速搭建[6]。从界面表现来看，活动管理、报名审核、公告发布和统计报表等页面都依赖组件库提供的布局与交互能力，从而保证风格统一、操作路径清晰和开发效率可控。

项目使用 Vite 作为前端构建与开发工具。Vite 通过现代化开发服务器和按需编译机制，能够显著提升启动速度和热更新体验[12]。在可视化方面，系统引入 Apache ECharts 实现活动数量、分类占比和服务时长等图表展示[7]，使管理员能够直观把握整体运行状况。这种“框架 + 组件库 + 工程化工具 + 图表库”的组合，是当前 Web 管理系统实践中较成熟的方案。

2.4 数据持久化与运行环境

在数据存储方面，系统同时具备 H2 与 MySQL 两种运行条件。H2 数据库体积小、配置简单、支持内存模式与文件模式，适合教学演示、自动化测试和本地快速验证[10]；MySQL 则适合后续将系统迁移到更稳定的持久化环境中[11]。项目当前在开发与测试中以 H2 为主，在配置层面保留了向 MySQL 切换的能力，这种设计兼顾了本地快速启动与后续扩展的需要。

值得注意的是，技术基础并不是简单堆叠框架名称，而是要与业务复杂度保持匹配。本系统没有引入微服务、消息队列或分布式缓存等更重的架构能力，而是围绕毕业设计场景选择单体式后端与组件化前端配合的方案。该方案能够在较短开发周期内保持结构清晰，且足以承载当前活动管理、流程校验和数据统计需

求。这种“适度设计”的思想，也是软件工程实践中非常重要的原则。

综上，本系统在技术路线选择上遵循“结构清晰、实现成熟、部署轻量、便于答辩演示”的原则。后端负责业务规则与数据一致性控制，前端负责交互展示与操作反馈，关系数据库负责业务数据存储与统计聚合，各类技术在本项目中分工明确、耦合适中，为系统实现提供了较为稳固的技术基础。

3 系统需求分析与数据库设计

3.1 角色需求与业务流程分析

根据任务书、开题报告和现有源码结构，本系统主要面向管理员与学生两类用户，但管理员在业务职责上还需要继续细分。系统管理员重点负责管理员档案维护、学生基础档案治理、公告治理、统计报表查看和运行审计；活动管理员重点负责志愿活动发布、报名审核、签到修正和服务时长确认；学生则承担活动浏览、在线报名、签到签退、消息查阅和个人信息维护等操作。当前实现层在 sys_user.role_code 中仍统一使用 ADMIN 编码承载后台账户，但在业务分析与模块设计层已经具备进一步细分管理员职责边界的条件。

在需求访谈与场景归纳层面，可以将高校志愿服务管理中的主要痛点概括为四类：第一，活动信息分散在微信群、表格和纸质表单中，学生难以及时获取统一入口；第二，报名审核与现场签到脱节，审核通过名单难以快速映射到签到结果；第三，服务时长往往事后手工汇总，缺乏可复核的过程明细；第四，管理者虽然掌握大量活动记录，却缺少便于比较与分析的报表视图。上述问题共同构成了本系统的直接建设动因。

表 3-1 角色与功能需求对应表

角色	主要功能	业务目标
系统管理员	管理员管理、学生管理、公告管理、统计报表、运行审计	保障系统基础数据完整、权限边界清晰和运行可控
活动管理员	活动管理、报名审核、签到管理、时长管理、活动过程治理	保障志愿活动执行效率和过程数据准确性
学生	活动浏览、活动报名、签到签退、时长查询、消息查看、个人信息维护	提升参与便利性和记录透明度

这种角色划分具有两个层面的意义：其一，从论文建模角度看，能够更清楚地说明后台并非单一角色，而是由不同岗位围绕统一平台协同完成管理；其二，从工程实现角度看，即使当前版本仍采用统一的 ADMIN 账户编码，也可以通过菜单、接口职责和页面分工体现系统管理员与活动管理员之间的权限边界，为后续细粒度授权扩展保留空间。

在业务流程上，系统围绕“活动全生命周期”展开设计。首先由管理员录入活动基本信息，发布后系统生成签到码与签退码；随后学生根据开放活动进行报名，管理员在后台审核报名申请；对审核通过的学生，系统预创建签到记录；学生在规定时间窗口内完成签到与签退后，系统生成待确认的服务时长记录；管理员确认后，系统同步更新学生累计服务时长，并通过消息中心向学生发送提醒。该流程将原本分散的线下步骤整合为统一闭环，提高了业务透明度。

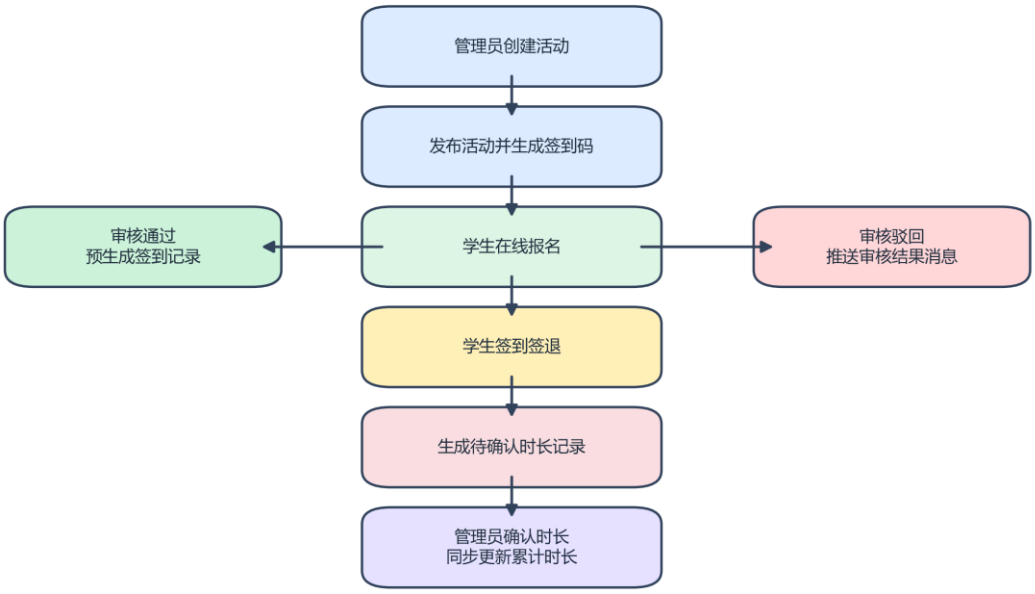

图 3-1 系统业务流程图

3.2 功能性与非功能性需求

从功能需求看，系统至少需要覆盖以下内容：一是用户认证与角色识别，保证不同角色获得正确的数据访问范围；二是活动管理与报名审核，保证活动信息录入完整、招募人数受控、审核状态可追踪；三是签到签退与时长认定，保证活动过程留痕与统计结果准确；四是公告消息和统计报表，保证管理者与学生能够及时获得业务反馈。

从非功能需求看，系统需要满足易用性、可靠性、安全性和可维护性等要求。易用性方面，页面应使用统一布局并提供明确反馈；可靠性方面，应通过参数校验、统一异常处理和状态约束减少非法操作；安全性方面，需要在登录认证、接口访问和数据存储层面建立基础保护；可维护性方面，则要求前后端目录结构清晰、模块职责明确、数据库表关系规范。由于该项目用于毕业设计场景，系统还需兼顾本地快速部署和答辩展示便利性。

3.3 数据需求分析与概念模型

结合业务流程，系统的数据对象主要包括学生、管理员、登录账号、活动分类、志愿活动、活动报名、活动签到、服务时长、公告、个人消息和操作日志等。上述实体之间形成了较清晰的关系链：sys_user 与 student/admin_user 之间为账号与档案的一对一关系；volunteer_activity 与 activity_signup、activity_sign、service_hours_record 之间表现为活动驱动的多实体联动关系；message_notice 与 notice 则承担公告分发和个人消息推送的作用。

在概念建模时，本文重点关注两个问题。第一，活动、报名、签到和时长之间必须形成可追踪的顺序关系，以支持后续状态判断和责任追溯；第二，学生累计服务时长不能直接人工维护，而应由已确认的服务时长记录自动汇总生成。基于这一思路，系统将 student.total_service_hours 设计为冗余汇总字段，同时通过 service_hours_record 保存每一次活动对应的明细时长，以保证查询效率与数据一致性之间的平衡。

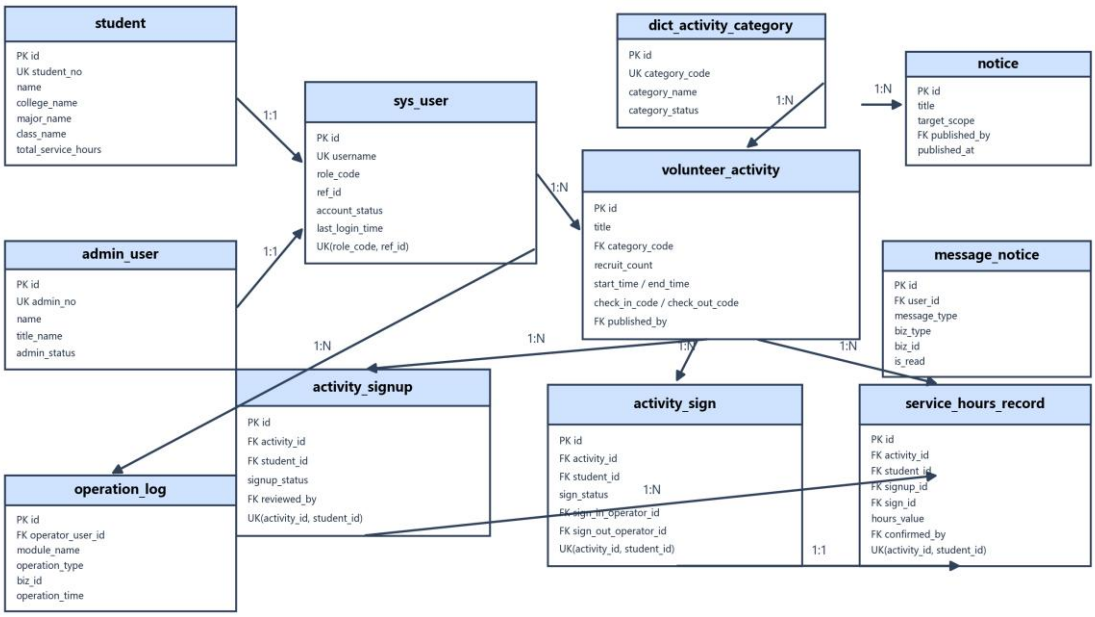

图 3-2 数据库 ER 图

3.4 逻辑结构设计

根据 `database/schema.sql` 和初始化数据脚本，系统当前共建立 11 张核心业务表，已经覆盖账户档案、活动主数据、活动过程、消息提醒和审计留痕五个层面。其中，student、admin_user 与 sys_user 支撑基础身份与登录关系；

dict_activity_category、volunteer_activity 与 notice 负责主数据维护；activity_signup、activity_sign 与 service_hours_record 负责活动过程建模；message_notice 与 operation_log 分别承担个人消息和审计留痕功能。各表普遍包含时间字段、状态字段与逻辑删除字段，以支持过程追踪、状态回溯和后续运维扩展。

从初始化脚本可见，系统预置了 2 个管理员账号、10 个学生账号、多个活动类别以及报名、签到、时长、公告和日志样本数据。这些数据并不是简单填充，而是围绕“管理员发布活动 - 学生报名 - 审核通过 - 现场签到 - 生成时长 - 推送消息”的业务链条构造，因而能够直接支撑学生首页、管理首页、活动管理页、时长管理页和统计报表页的运行展示。数据库设计与页面呈现之间形成了较强的可验证联动关系。

表 3-2 核心数据表分层概览

数据层级	包含数据表	设计重点
账户与档案层	student、admin_user、sys_user	建立学生、管理员与登录账户之间的映射关系
主数据层	dict_activity_category、volunteer_activity、notice	保存活动分类、活动主体和公告主体信息
过程数据层	activity_signup、activity_sign、service_hours_record	承载报名、签到签退和时长确认全过程数据
消息与审计层	message_notice、operation_log	保存个人消息提醒和后台关键操作日志

这种设计的优点在于：一方面通过拆分主数据和过程数据，使活动状态、报名状态、签到状态和时长状态的表达更加清晰；另一方面通过外键约束、唯一约束和状态字段，将“一个学生针对一个活动只能形成一条报名、一条签到主记录、一条时长记录”的业务规则下沉到数据库层，有效减少重复提交或异常调用导致的数据不一致风险。以下将对 11 张核心数据表逐表说明其字段类型、主外键约束及业务用途。

3.4.1 账户与基础档案表设计

账户与档案层是系统权限控制和业务归属的基础。student、admin_user 与 sys_user 三张表共同完成“业务档案 + 登录账号 + 角色映射”的建模方式，既适合当前毕业设计项目的简化实现，也便于后续向更细粒度权限体系扩展。

student 表对应学生档案，是学生管理、报名审核、签到签退、时长汇总和个人中心页面的数据基础。其字段除基本身份信息外，还保存 total_service_hours 冗余汇总值，以支持首页和报表的快速查询。

表 3-3 student 表结构设计

字段名	数据类型	键/约束	字段说明
id	BIGINT	主键，AUTO_INCREMENT	学生档案主键
student_no	VARCHAR(50)	NOT NULL，UNIQUE	学号，作为学生唯一业务编号
name	VARCHAR(100)	NOT NULL	学生姓名
gender	VARCHAR(20)	无	性别信息
college_name	VARCHAR(100)	NOT NULL	所属学院名称
major_name	VARCHAR(100)	NOT NULL	所属专业名称
class_name	VARCHAR(100)	NOT NULL	所属班级名称
phone	VARCHAR(20)	无	联系电话
email	VARCHAR(100)	无	邮箱地址
avatar_url	VARCHAR(255)	无	头像文件访问路径
total_service_hours	DECIMAL(6,2)	DEFAULT 0.00	累计已确认志愿服务时长
student_status	VARCHAR(20)	DEFAULT 'NORMAL'	学生状态，如正常、停用
remark	VARCHAR(255)	无	备注信息
created_at	DATETIME	NOT NULL	创建时间
updated_at	DATETIME	NOT NULL	更新时间
is_deleted	TINYINT	DEFAULT 0	逻辑删除标记

admin_user 表存储后台管理员档案，对应管理员管理页面中的维护对象。结合初始化数据可见，系统当前已存在“系统管理员”和“学院管理员”两类后台人员，这为论文中将管理员细分为系统管理员与活动管理员提供了现实依据。

表 3-4 admin_user 表结构设计

字段名	数据类型	键/约束	字段说明
id	BIGINT	主键，AUTO_INCREMENT	管理员档案主键
admin_no	VARCHAR(50)	NOT NULL，	管理员编号

		UNIQUE	
name	VARCHAR(100)	NOT NULL	管理员姓名
phone	VARCHAR(20)	无	联系电话
email	VARCHAR(100)	无	邮箱地址
title_name	VARCHAR(100)	无	岗位或职务名称
admin_status	VARCHAR(20)	DEFAULT 'NORMAL'	管理员状态
created_at	DATETIME	NOT NULL	创建时间
updated_at	DATETIME	NOT NULL	更新时间
is_deleted	TINYINT	DEFAULT 0	逻辑删除标记

sys_user 表是统一登录账户表，对应登录页、认证授权逻辑和菜单鉴权逻辑。当前实现层使用 ADMIN 与 STUDENT 两种 role_code，但在业务设计层可进一步细分 ADMIN 账户承担系统管理与活动治理两类职责，且二者共享同一账号框架。

表 3-5 sys_user 表结构设计

字段名	数据类型	键/约束	字段说明
id	BIGINT	主键，AUTO_INCREMENT	系统账户主键
username	VARCHAR(50)	NOT NULL，UNIQUE	登录用户名
password_hash	VARCHAR(128)	NOT NULL	密码摘要
role_code	VARCHAR(20)	NOT NULL	角色编码，如 ADMIN、STUDENT
ref_id	BIGINT	NOT NULL	关联 student 或 admin_user 的主键
account_status	VARCHAR(20)	DEFAULT 'ENABLED'	账户状态
last_login_time	DATETIME	无	最近登录时间
created_at	DATETIME	NOT NULL	创建时间
updated_at	DATETIME	NOT NULL	更新时间
is_deleted	TINYINT	DEFAULT 0	逻辑删除标记
(role_code, ref_id)	联合唯一键	UNIQUE	限制同一档案在同一角色下只能绑定一个登录账户

3.4.2 活动主数据与通知表设计

活动主数据层决定了学生端浏览内容和管理端治理对象的范围。dict_activity_category、volunteer_activity 与 notice 三张表分别负责活动分类、活动主档案和公告通知，为后续报名、签到和消息下发提供主数据来源。

dict_activity_category 表是活动分类字典表，为活动发布页面提供类别下拉数据，也为统计报表中的分类分布分析提供维度基础。

表 3-6 dict_activity_category 表结构设计

字段名	数据类型	键/约束	字段说明
id	BIGINT	主键，AUTO_INCREMENT	活动分类主键
category_code	VARCHAR(20)	NOT NULL，UNIQUE	分类编码
category_name	VARCHAR(100)	NOT NULL	分类名称
sort_no	INT	DEFAULT 0	排序号
category_status	VARCHAR(20)	DEFAULT 'ENABLED'	分类启用状态
created_at	DATETIME	NOT NULL	创建时间
updated_at	DATETIME	NOT NULL	更新时间

volunteer_activity 表是系统的核心主表，直接对应活动管理页、学生活动列表页、活动详情页、首页统计卡片和报表数据源。活动发布时间、签到码、签退码等关键字段均保存在该表中，体现了活动从草稿到发布再到执行的完整生命周期。

表 3-7 volunteer_activity 表结构设计

字段名	数据类型	键/约束	字段说明
id	BIGINT	主键，AUTO_INCREMENT	活动主键
title	VARCHAR(100)	NOT NULL	活动标题
category_code	VARCHAR(20)	NOT NULL，FK	关联 dict_activity_category.category_code
location	VARCHAR(255	NOT NULL	活动地点

	)		
organizer_name	VARCHAR(100)	NOT NULL	主办方名称
description	TEXT	无	活动详细说明
cover_url	VARCHAR(255)	无	活动封面图地址
attachment_url	VARCHAR(255)	无	活动附件地址
recruit_count	INT	NOT NULL	计划招募人数
signup_deadline	DATETIME	NOT NULL	报名截止时间
start_time	DATETIME	NOT NULL	活动开始时间
end_time	DATETIME	NOT NULL	活动结束时间
service_hours	DECIMAL(6,2)	NOT NULL	活动对应服务时长
check_in_code	VARCHAR(20)	无	签到码
check_out_code	VARCHAR(20)	无	签退码
activity_status	VARCHAR(20)	DEFAULT 'DRAFT'	活动状态，如草稿、已发布、已取消
published_by	BIGINT	FK	关联 sys_user.id，记录发布人
published_at	DATETIME	无	发布时间
created_at	DATETIME	NOT NULL	创建时间
updated_at	DATETIME	NOT NULL	更新时间
is_deleted	TINYINT	DEFAULT 0	逻辑删除标记

notice 表存储公告主体信息，对应管理端公告管理页面与学生端首页公告区域。该表与 message_notice 配合使用，形成“公告主表 + 面向个人的消息分发表”的通知机制。

表 3-8 notice 表结构设计

字段名	数据类型	键/约束	字段说明
id	BIGINT	主键，AUTO_INCREMENT	公告主键
title	VARCHAR(100)	NOT NULL	公告标题
content	TEXT	NOT NULL	公告正文
attachment_url	VARCHAR(255)	无	附件地址
target_scope	VARCHAR(20)	DEFAULT	目标范围

publish_status	VARCHAR(20)	'ALL_STUDENTS' DEFAULT 'PUBLISHED'	发布状态
published_by	BIGINT	NOT NULL，FK	关联 sys_user.id，记录发布人
published_at	DATETIME	NOT NULL	发布时间
created_at	DATETIME	NOT NULL	创建时间
updated_at	DATETIME	NOT NULL	更新时间
is_deleted	TINYINT	DEFAULT 0	逻辑删除标记

3.4.3 活动过程数据表设计

活动过程层负责承载“报名 - 审核 - 签到 - 签退 - 时长确认 - 消息提醒 - 审计留痕”的全过程数据，是数据库设计中最能体现业务闭环的部分。该层的外键关系最为密集，也是页面联动最直接的区域。

activity_signup 表记录学生报名行为与管理员审核结果，对应学生端活动详情/我的报名页面和管理端报名审核页面。表中通过唯一约束限制同一学生对同一活动只能形成一条报名记录。

表 3-9 activity_signup 表结构设计

字段名	数据类型	键/约束	字段说明
id	BIGINT	主键，AUTO_INCREMENT	报名记录主键
activity_id	BIGINT	NOT NULL，FK	关联 volunteer_activity.id
student_id	BIGINT	NOT NULL，FK	关联 student.id
signup_status	VARCHAR(20)	DEFAULT 'PENDING'	报名状态，如待审、通过、驳回、取消
review_comment	VARCHAR(255)	无	审核意见
reviewed_by	BIGINT	FK	关联 sys_user.id，记录审核人
reviewed_at	DATETIME	无	审核时间
cancel_reason	VARCHAR(255)	无	取消原因
signup_time	DATETIME	NOT NULL	报名时间
created_at	DATETIME	NOT NULL	创建时间
updated_at	DATETIME	NOT NULL	更新时间

is_deleted	TINYINT	DEFAULT 0	逻辑删除标记
(activity_id, student_id)	联合唯一键	UNIQUE	限制重复报名

activity_sign 表记录学生签到签退过程，是学生端签到签退页面和管理端签到管理页面的直接数据来源。系统在报名审核通过后预先生成该表记录，使学生资格控制与现场签到流程保持一致。

表 3-10 activity_sign 表结构设计

字段名	数据类型	键/约束	字段说明
id	BIGINT	主键，AUTO_INCREMENT	签到记录主键
activity_id	BIGINT	NOT NULL，FK	关联 volunteer_activity.id
student_id	BIGINT	NOT NULL，FK	关联 student.id
sign_status	VARCHAR(20)	DEFAULT 'UNSIGNED'	签到状态，如未签到、已签到、已签退、管理员补录
sign_in_time	DATETIME	无	签到时间
sign_out_time	DATETIME	无	签退时间
sign_in_mode	VARCHAR(20)	无	签到方式，如学生输入或管理员补录
sign_out_mode	VARCHAR(20)	无	签退方式
sign_in_operator_id	BIGINT	FK	关联 sys_user.id，记录签到操作人
sign_out_operator_id	BIGINT	FK	关联 sys_user.id，记录签退操作人
exception_remark	VARCHAR(255)	无	异常说明
created_at	DATETIME	NOT NULL	创建时间
updated_at	DATETIME	NOT NULL	更新时间
(activity_id, student_id)	联合唯一键	UNIQUE	限制一个活动只有一条签到主记录

service_hours_record 表记录服务时长明细及确认状态，对应学生端我的时长页面、管理端时长管理页面和统计报表中的时长聚合逻辑。该表是 total_service_hours 汇总值的依据，因此在数据设计上同时关联活动、报名与签到

记录。

表 3-11 service_hours_record 表结构设计

字段名	数据类型	键/约束	字段说明
id	BIGINT	主键，AUTO_INCREMENT	时长记录主键
activity_id	BIGINT	NOT NULL，FK	关联 volunteer_activity.id
student_id	BIGINT	NOT NULL，FK	关联 student.id
signup_id	BIGINT	FK	关联 activity_signup.id
sign_id	BIGINT	FK	关联 activity_sign.id
hours_value	DECIMAL(6,2)	NOT NULL	本次活动产生的服务时长
hours_status	VARCHAR(20)	DEFAULT 'PENDING_CONFIRM'	时长状态，如待确认、已确认、已撤销
generated_at	DATETIME	NOT NULL	时长记录生成时间
confirmed_by	BIGINT	FK	关联 sys_user.id，记录确认人
confirmed_at	DATETIME	无	确认时间
remark	VARCHAR(255)	无	补充说明
created_at	DATETIME	NOT NULL	创建时间
updated_at	DATETIME	NOT NULL	更新时间
(activity_id, student_id)	联合唯一键	UNIQUE	限制同一学生在同一活动下只有一条时长记录

message_notice 表保存面向个人的通知消息，是学生端消息中心页面的数据基础。报名审核结果、时长确认提醒和公告分发结果均可落到该表中，体现了系统“事件驱动消息通知”的设计思路。

表 3-12 message_notice 表结构设计

字段名	数据类型	键/约束	字段说明
id	BIGINT	主键，AUTO_INCREMENT	消息主键
user_id	BIGINT	NOT NULL，FK	关联 sys_user.id，指向接收消息的登录账

			户
message_type	VARCHAR(50)	NOT NULL	消息类型
title	VARCHAR(100)	NOT NULL	消息标题
content	VARCHAR(255)	NOT NULL	消息内容摘要
biz_type	VARCHAR(50)	无	业务类型，如活动、时长、公告
biz_id	BIGINT	无	关联业务主键
is_read	TINYINT	DEFAULT 0	已读标记
created_at	DATETIME	NOT NULL	创建时间
read_at	DATETIME	无	阅读时间

operation_log 表记录后台关键操作，是系统审计与问题追踪的重要依据。当前源码已在示例数据中保留活动发布与时长确认日志，后续可继续扩展到管理员管理、公告发布和异常补录等更多治理行为。

表 3-13 operation_log 表结构设计

字段名	数据类型	键/约束	字段说明
id	BIGINT	主键，AUTO_INCREMENT	日志主键
operator_user_id	BIGINT	NOT NULL，FK	关联 sys_user.id，记录操作账户
operator_role_code	VARCHAR(20)	NOT NULL	操作角色编码
module_name	VARCHAR(50)	NOT NULL	所属功能模块
operation_type	VARCHAR(50)	NOT NULL	操作类型
biz_id	BIGINT	无	关联业务主键
request_path	VARCHAR(255)	无	请求路径
operation_desc	VARCHAR(255)	无	操作说明
operation_time	DATETIME	NOT NULL	操作时间

4 系统架构与功能模块设计

4.1 系统总体架构设计

结合源码结构，本系统按照“表示层、业务层、数据访问层、数据存储层”进行分层组织。表示层由 Vue 3 页面与组件构成，负责表单录入、列表展示、图表渲染和操作反馈；业务层由 Spring Boot 服务类承载，负责报名审核、签到规则判断、时长确认与消息生成等核心逻辑；数据访问层通过 MyBatis-Plus Mapper 实现实体查询和状态更新；数据存储层则由 H2/MySQL 维护结构化数据与本地上传文件目录。分层设计使界面更新、业务规则调整和数据结构修改能够相对独立地进行。

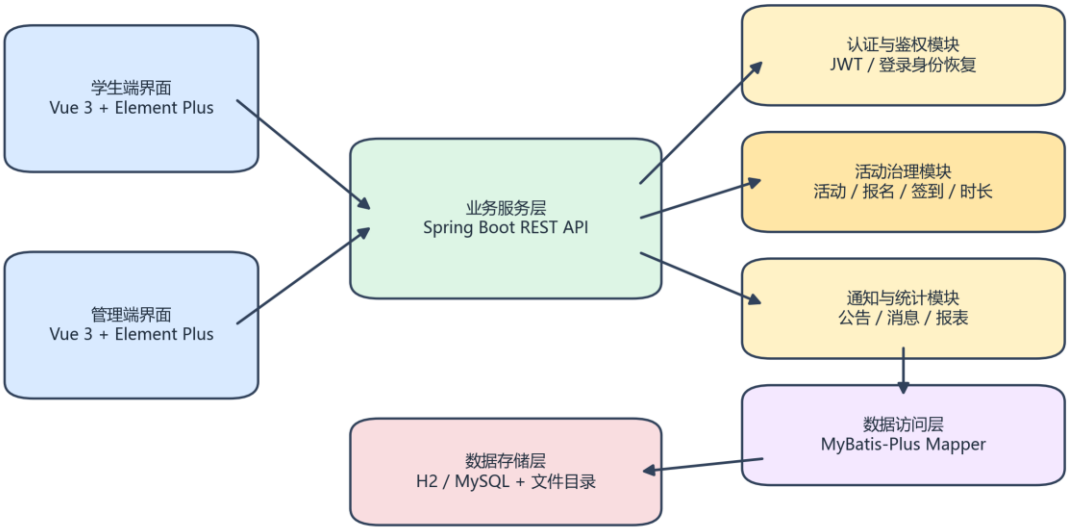

图 4-1 系统总体架构图

在接口分层上，系统将认证、学生端和管理员端能力进行隔离。认证模块仅处理登录与当前用户信息；学生端围绕活动浏览、个人报名、签到签退和消息查询展开；管理员端则提供学生管理、活动管理、审核处理、公告发布和统计报表。通过这种模块边界划分，前端路由与后端接口能够形成稳定映射，有利于按角色组织菜单和页面。

4.2 功能模块设计

在模块设计层面，系统不再简单地将后台视为单一管理员入口，而是将管理员细分为系统管理员和活动管理员两类岗位角色。当前实现中二者共用 sys_user 的 ADMIN 角色编码和 admin_user 档案结构，但在业务职责上已经可以明确区

分：系统管理员偏向账户、档案、公告、报表和审计治理；活动管理员偏向活动发布、报名审核、签到补录和时长确认等活动执行治理。这种“技术上统一账号模型、业务上区分职责边界”的设计，更符合高校多岗位协同管理的实际情况。

表 4-1 管理员细分职责设计

角色	核心职责	对应页面/模块	主要涉及数据表
系统管理员	管理员维护、学生档案维护、公告治理、报表查看、运行审计	管理员管理、学生管理、公告管理、统计报表	admin_user、sys_user、student、notice、message_notice、operation_log
活动管理员	活动创建发布、报名审核、签到修正、时长确认、活动过程治理	活动管理、报名审核、签到管理、时长管理	volunteer_activity、activity_signup、activity_sign、service_hours_record
学生	浏览活动、在线报名、签到签退、查看时长与消息、维护个人资料	学生首页、活动列表、我的报名、签到签退、我的时长、我的消息、个人中心	student、volunteer_activity、activity_signup、activity_sign、service_hours_record、message_notice

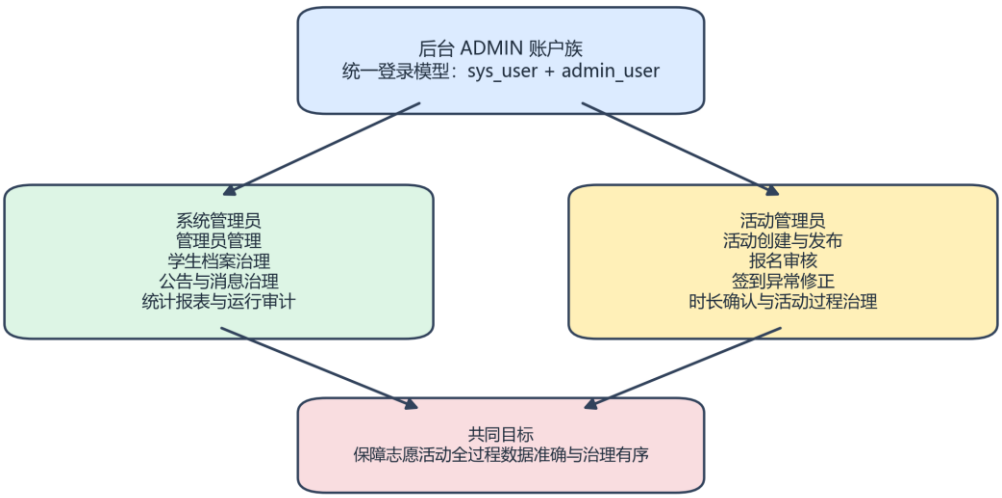

图 4-2 管理员职能细分图

学生端模块围绕“浏览活动、参与活动、沉淀个人记录”三条主线展开设计。学生首先在首页获取公告、近期活动和累计时长摘要，然后进入活动列表和活动详

情完成报名，接着在签到签退模块完成现场操作，最后通过我的报名、我的时长和我的消息查看系统反馈。个人中心模块则负责维护手机、邮箱、头像等基础档案字段，使个人资料维护与活动过程数据保持合理分离。

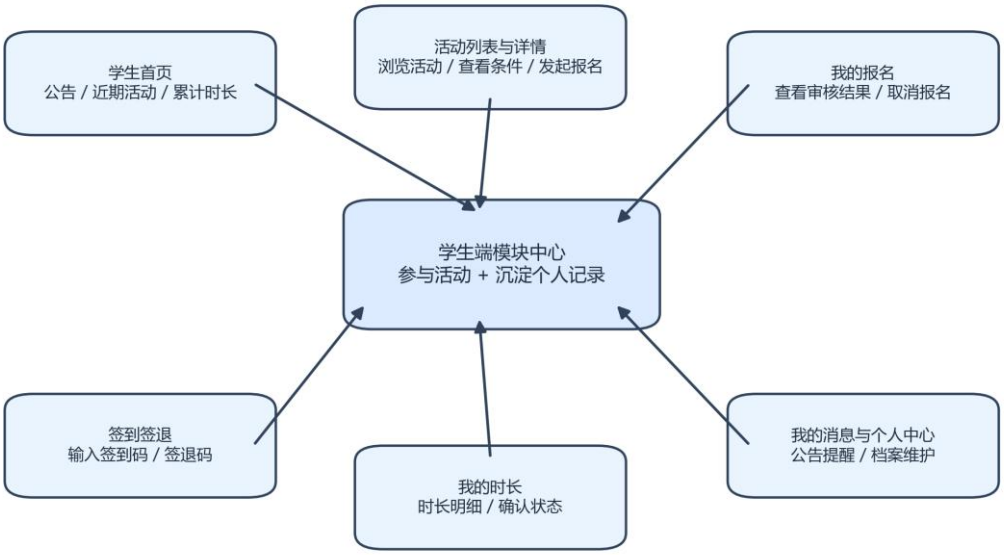

图 4-3 学生端功能模块设计图

管理端模块则围绕“主数据治理、过程治理、结果分析”三层结构展开。管理首页负责概览统计；学生管理和管理员管理负责基础档案治理；活动管理、报名审核、签到管理和时长管理构成活动过程治理链路；公告管理和统计报表则承担信息发布与决策分析功能。这样划分后，不同岗位管理员可以在统一后台框架下沿着相对独立的菜单路径完成各自工作。

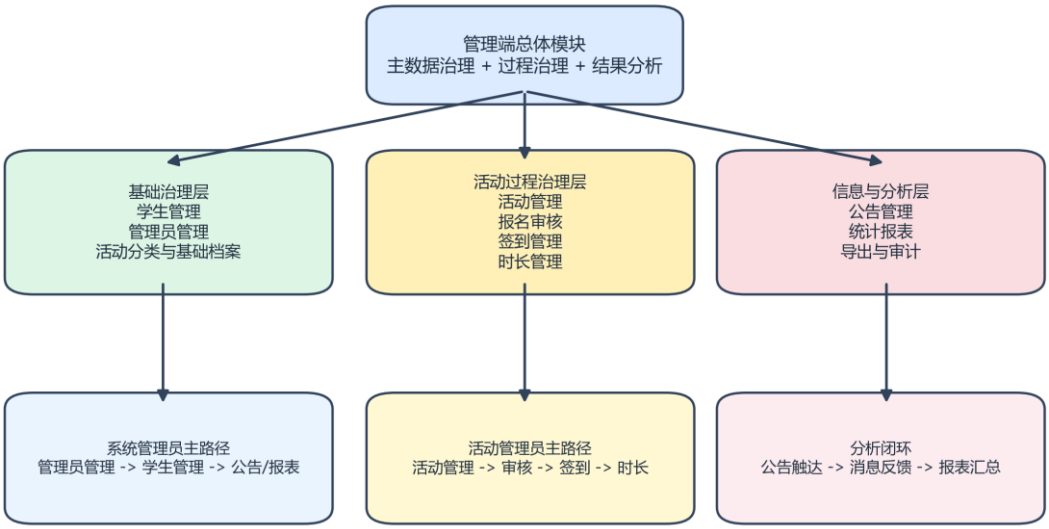

图 4-4 管理端功能模块设计图

4.3 关键业务机制设计

为了更准确评估活动招募执行情况，本文定义活动招募完成率 η_j，用于表示第 j 个活动的审核通过人数与计划招募人数之间的比例，其表达式如下：

$$\eta_j = \frac{N_{approved,j}}{Q} \times 100\% \tag{4-1}$$

其中，N_approved,j 表示第 j 个活动已审核通过的人数，Q_j 表示该活动计划招募人数。该指标可用于后台报表中的活动组织效果分析，也可为后续活动扩招、补招或提前截止报名提供决策参考。

对于学生累计服务时长，系统采用基于明细记录汇总的方式进行计算，而非直接在活动完成时对学生总时长进行简单累加。设学生已确认的服务时长记录共有 n 条，第 i 条记录的服务时长为 h_i，确认状态指示变量为 δ_i，则累计时长 H_total 的计算如下：

$$H_{total} = \Sigma_{i=1}^{n} h_i \cdot \delta_i \tag{4-2}$$

当第 i 条记录状态为已确认时，δ_i 取 1；当状态为待确认或已撤销时，δ_i 取 0。采用该模型能够避免手工修正带来的累计误差，并保证在撤销时长后重新汇总即可恢复一致结果，这与源码中 refreshStudentTotalHours 的实现方式完全一致。

此外，系统对签到签退时间窗口进行了显式约束：签到允许在活动开始前 30 分钟至开始后 30 分钟内完成，签退允许在活动结束前 30 分钟至结束后 120 分

钟内完成。该规则既保留了现场活动组织的灵活性，又能避免过早或过晚操作影响时长认定，是系统业务规则设计中的关键部分。

4.4 前后端协同与交互设计

在前后端协同机制上，前端以路由组织业务入口，通过 Axios 向后端发送 HTTP 请求，并在请求头中携带登录令牌；后端根据 JWT 中的角色信息恢复当前用户身份，分别执行学生或管理员权限校验。界面层面，表单页重点强调录入完整性与即时反馈，列表页强调筛选、分页和状态展示，统计页则强调图形化表达。这样的交互安排能够较好适配毕业设计答辩场景下的演示需求，也使数据库主外键关系能够在页面层获得清晰映射。

为了体现“页面 - 服务 - 数据表”的联动关系，本文进一步将主要页面与后端服务方法、核心数据表对应起来。由此可以看出，本系统并非孤立地堆叠页面，而是围绕业务闭环建立了从界面操作到服务处理再到数据落库的完整映射链路。

从源码实现看，登录与账户初始化主要由 AuthService.login 完成；学生管理与管理员管理分别由 AdminPortalService.listStudents/saveStudent 和 listAdmins/saveAdmin 承担；活动治理主链路对应 AdminPortalService.saveActivity、publishActivity、approveSignup、fixSign、confirmHours 等方法；学生端活动参与主链路则对应 StudentPortalService.listActivities、applyActivity、signIn 和 signOut。上述方法最终都会落到相应主表或过程表中，从而形成前端页面、业务服务与数据库之间的一一映射。

表 4-2 页面、服务与数据库联动关系表

功能域	典型页面	核心数据表	联动结果
账户与档案	登录页、学生管理页、管理员管理页	sys_user、student、admin_user	完成身份校验、档案维护和账户绑定
活动主数据	活动管理页、活动列表页、活动详情页	volunteer_activity、dict_activity_category	完成活动创建、发布、展示与分类筛选
报名审核	活动详情页、我的报名页、报名审核页	activity_signup、activity_sign	审核通过后预生成签到主记录
签到与时长	签到签退页、时长管理页、我的时长页	activity_sign、service_hours_record、student	完成签到流转、时长确认与累计回写
公告与消息	公告管理页、学生首	notice、message_notice	形成公告主体与个人

	页、我的消息页		消息分发
统计与审计	管理首页、统计报表页	volunteer_activity、service_hours_record、operation_log	输出统计分析结果并保留关键操作日志

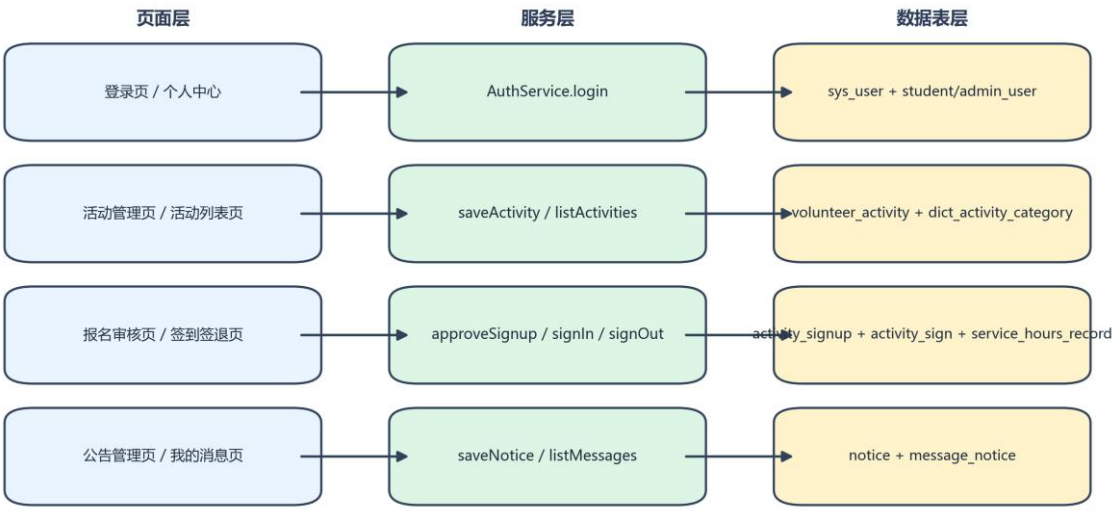

图 4-5 页面、服务与数据库联动图

5 系统实现与关键技术分析

5.1 认证授权实现

认证流程由 AuthService 与 JwtTokenProvider 共同完成。用户在登录页输入账号和密码后，后端首先根据用户名和账户状态查询 sys_user，再利用 PasswordUtils 对输入密码进行摘要匹配；校验通过后，系统记录最近登录时间，并构建包含 userId、username、roleCode 与 refId 的登录对象，最终生成 JWT 令牌返回前端。该设计既满足了教学场景对轻量认证的需求，也为后续角色识别提供了统一依据。

代码 5-1 JWT 令牌生成核心实现

```
public String generate(LoginUser loginUser) {
    Instant now = Instant.now();
    Instant expireAt = now.plusSeconds(expirationMinutes * 60);
    return Jwts.builder()
        .subject(String.valueOf(loginUser.getUserId()))
        .claim("username", loginUser.getUsername())
        .claim("roleCode", loginUser.getRoleCode())
        .claim("refId", loginUser.getRefId())
        .issuedAt(Date.from(now))
        .expiration(Date.from(expireAt))
        .signWith(secretKey)
        .compact();
}
```

从实现细节看，JWT 负载中保留了最小必要信息，避免每次接口调用都重新查询全部用户数据。与此同时，SecurityHelper 在业务层对角色进行二次校验，保证管理员接口与学生接口的权限边界清晰。该方案结构简单、易于演示，适合本课题的实现深度。

除了认证逻辑本身，登录后的用户资料构建同样值得关注。AuthService 在返回登录结果时，会根据 sys_user 中保存的 roleCode 和 refId，分别查询 student 或 admin_user 档案信息，并组装 displayName、collegeName 等前端直接可用的数据结构。这样做的好处是，前端无需在登录后再次发起多次补充请求即可完成界面初始化，能够有效简化状态管理逻辑。

图 5-1 系统登录界面

5.2 活动管理与报名审核实现

管理员对活动的创建、编辑和发布主要在 AdminPortalService 中实现。系统在保存活动时首先校验报名截止时间、开始时间和结束时间之间的逻辑关系，防止产生非法时间配置；当活动被正式发布时，系统自动生成签到码与签退码，并写入发布时间、发布人信息。这一机制将“活动信息维护”与“活动进入可报名状态”明确区分开来，有助于避免草稿信息过早暴露给学生。

代码 5-2 活动发布与签到码生成实现

```
@Transactional
public void publishActivity(Long activityId) {
    VolunteerActivity activity = mustActivity(activityId);
    activity.setActivityStatus("PUBLISHED");
    activity.setCheckInCode(randomCode());
    activity.setCheckOutCode(randomCode());
    activity.setPublishedAt(LocalDateTime.now());
    activity.setPublishedBy(SecurityHelper.currentUser().getUserId());
    volunteerActivityMapper.updateById(activity);
}
```

报名审核的核心目标是控制名额与保证后续签到链路完整。当管理员审核通过某条报名记录后，系统不仅更新其状态，还会立即为该学生和该活动创建一条初始状态为 UNSIGNED 的签到记录。这样一来，学生是否有资格签到，不再依赖临时判断，而是直接依附于审核结果生成的业务对象，使后续签到、签退和时长生成过程更容易保持一致。

当活动被取消时，后台还会触发一组联动处理：首先修改活动状态，其次向所有已报名学生发送活动变更消息，最后将已有且未撤销的服务时长记录统一标

记为 REVOKED，并重新汇总学生累计时长。这一设计说明系统并未只关注“正常流程”，而是同时考虑了活动异常终止、补录和撤销等实际管理场景，增强了系统逻辑的完整性。

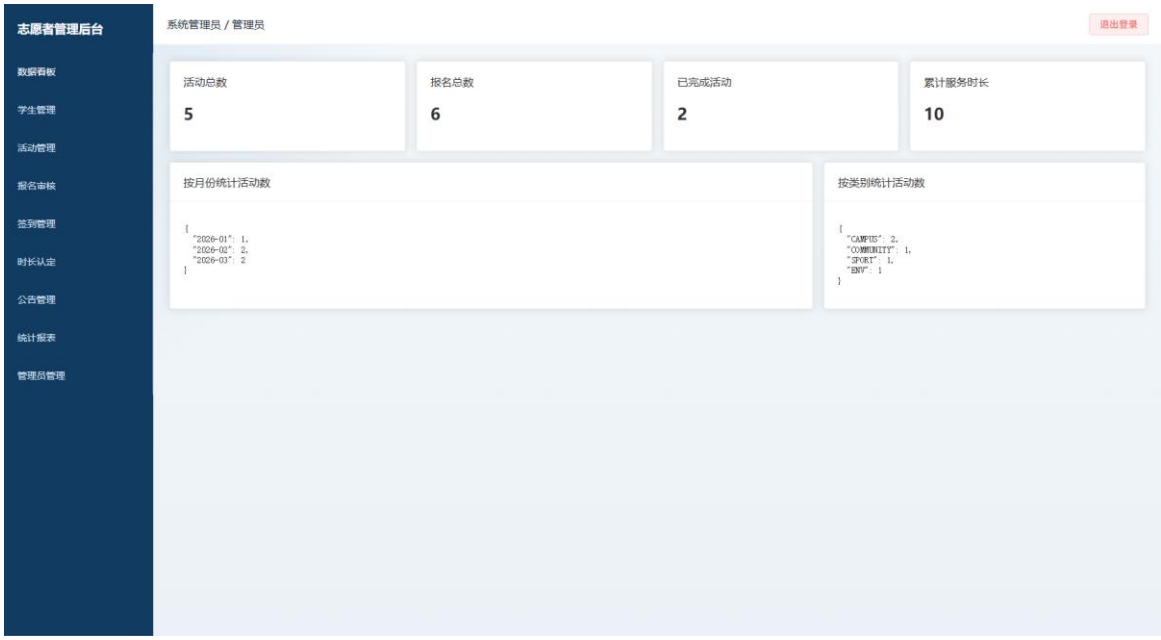

图 5-2 管理端首页

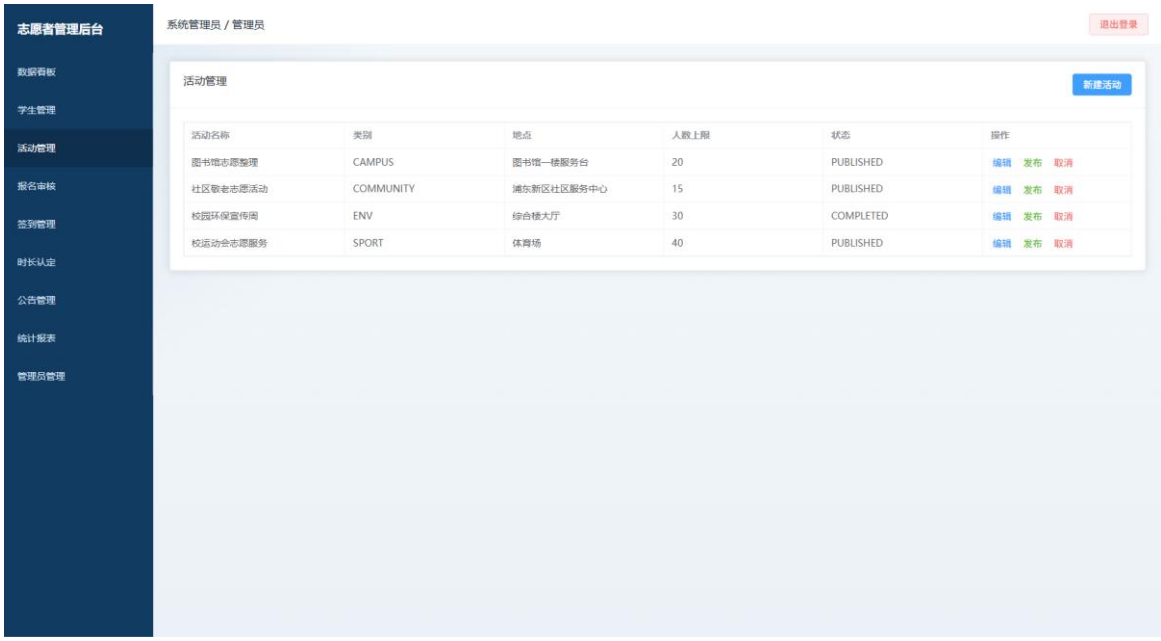

图 5-3 活动管理界面

5.3 学生报名、签到签退与时长生成实现

学生端围绕“报名前校验、活动中签到、活动后生成时长”三个阶段展开。applyActivity 方法在报名时会检查活动状态、报名截止时间和剩余名额，同时避免同一学生重复报名；signIn 与 signOut 方法则分别校验时间窗口、签到码正确

性与当前签到状态，防止无效操作。业务上，只有审核通过且存在签到记录的学生，才可以进入签到签退流程。

代码 5-3 学生签退与服务时长生成实现

```
@Transactional
public void signOut(SignCodeRequest request) {
    Student student = currentStudent();
    VolunteerActivity activity = getActivity(request.activityId());
    ActivitySign sign = getSignRecord(student.getId(), request.activityId());
    if (!Objects.equals(activity.getCheckOutCode(), request.code())) {
        throw new BusinessException("签退码错误");
    }
    sign.setSignStatus("SIGNED_OUT");
    sign.setSignOutTime(LocalDateTime.now());
    activitySignMapper.updateById(sign);
    generateHoursRecord(student.getId(), activity, sign);
}
```

generateHoursRecord 方法在检测到学生完成签退后，为其生成一条待确认的服务时长记录，并携带活动编号、学生编号、报名编号和签到编号。该记录并不会直接写入学生累计时长，而是等待管理员在后台确认。通过这种“先生成明细、后人工确认、再统一汇总”的设计，系统既兼顾了自动化处理效率，也保留了对异常情况进行人工复核的空间。

在学生端个人功能实现方面，系统还提供了个人资料维护与消息查看能力。学生可以修改手机、邮箱、头像和备注等基本信息，这些字段与活动参与主链路分离，既保证了档案管理灵活性，也避免对关键统计数据产生干扰。消息列表则通过 message_notice 统一承载报名审核结果、公告提醒和时长确认信息，使学生能够在同一入口查看个人相关通知，减少信息遗漏。

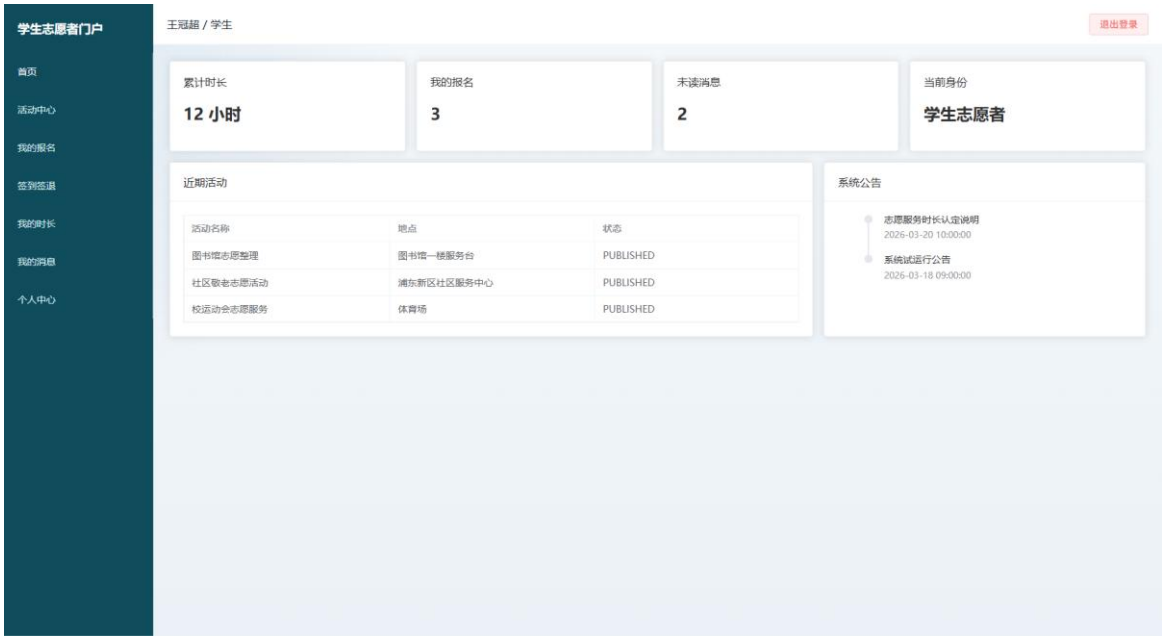

图 5-4 学生端首页

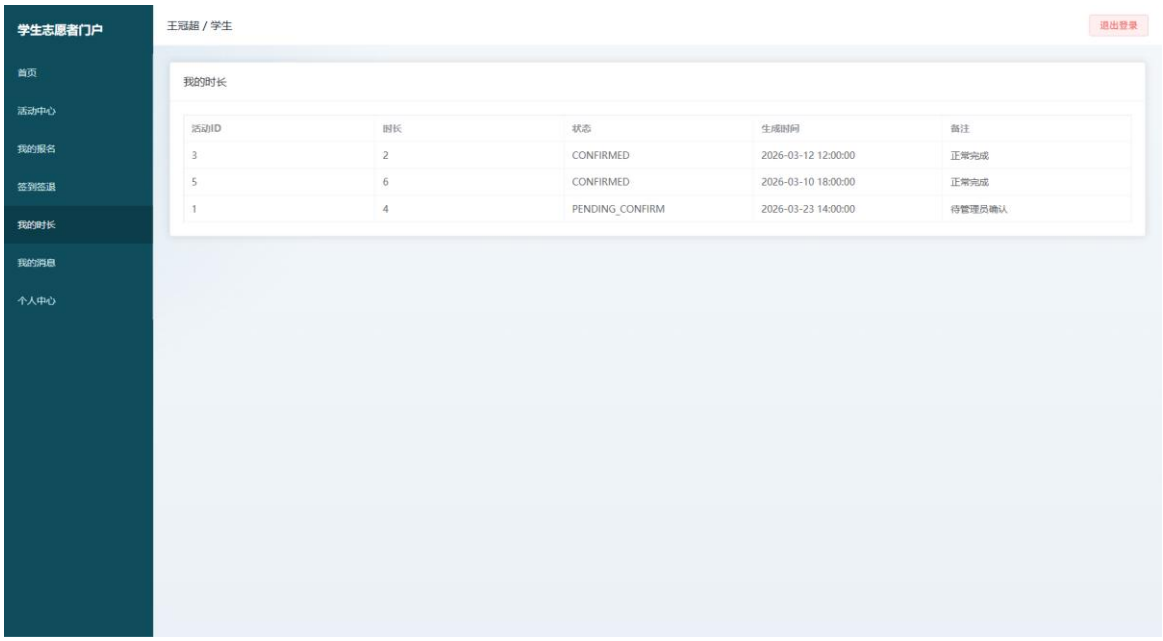

图 5-5 我的时长界面

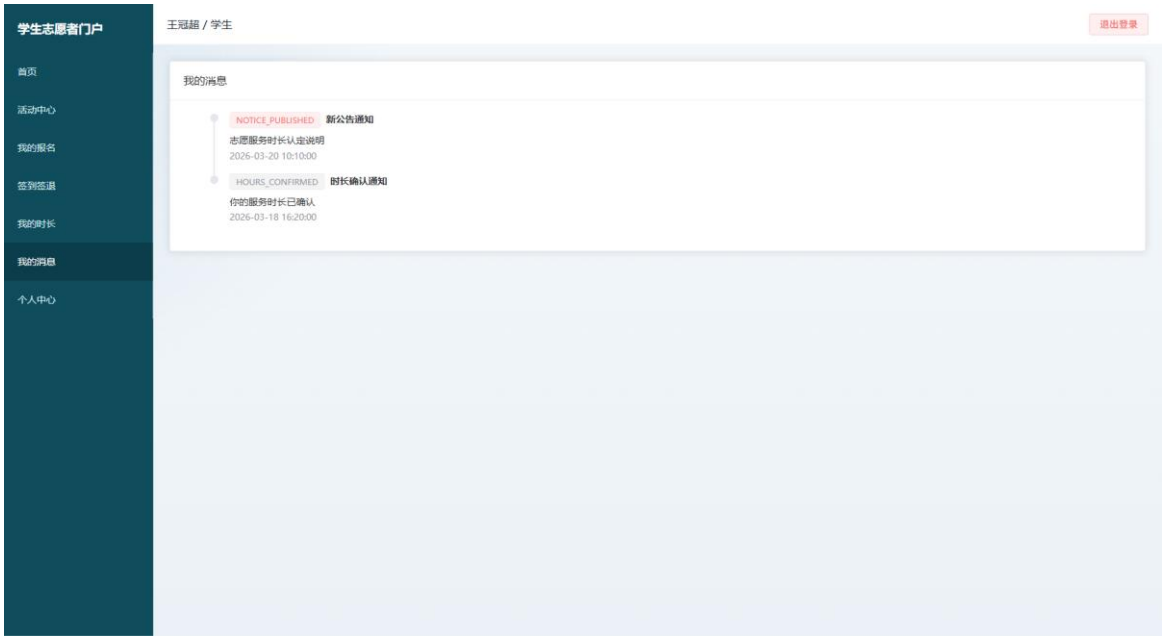

图 5-6 我的消息界面

5.4 公告推送与统计报表实现

公告功能由管理员统一发布。系统保存公告主体信息后，会遍历所有启用中的学生账号，为其批量生成个人消息记录，从而实现“公共公告 + 个人消息”双通道通知。这样既能满足首页公告展示的需要，也能让学生在个人消息列表中查看历史提醒，提高通知触达率。

在统计报表方面，系统根据活动、报名、签到和时长四类数据进行聚合。后台首页 dashboard 方法统计活动总数、报名总数、已完成活动数与累计服务时长，并进一步按月份、活动类别和学院维度计算分布数据；activityReport、studentHoursReport 和 monthlyReport 则分别面向活动、学生与月份输出结构化统计结果。相较于简单列表查询，这种聚合式设计更适合管理决策与答辩展示。

同时，系统提供 CSV 导出能力，以满足管理者对线下归档或进一步分析的需要。导出逻辑并未额外依赖复杂的报表中间件，而是将聚合结果直接格式化为文本内容返回，包含活动名称、分类、地点、报名人数、签到人数、完成人数和累计时长等关键字段。对于毕业设计项目而言，这种实现方式成本较低、可读性较强，也能清晰展示后端如何将统计对象转换为最终可交付数据。

代码 5-4 后台统计聚合实现片段

```
Map<String, Long> monthCountMap =
activities.stream().collect(Collectors.groupingBy(
    activity -> YearMonth.from(activity.getStartTime()).toString(),
    LinkedHashMap::new,
    Collectors.counting()
```

```
));
Map<String, BigDecimal> collegeHoursMap = studentMapper.selectList(new
LambdaQueryWrapper<Student>())
    .stream()
    .collect(Collectors.groupingBy(
        Student::getCollegeName,
        LinkedHashMap::new,
        Collectors.mapping(
            student -> student.getTotalServiceHours() == null ? BigDecimal.ZERO :
student.getTotalServiceHours(),
            Collectors.reducing(BigDecimal.ZERO, BigDecimal::add)
        )
    ));
```

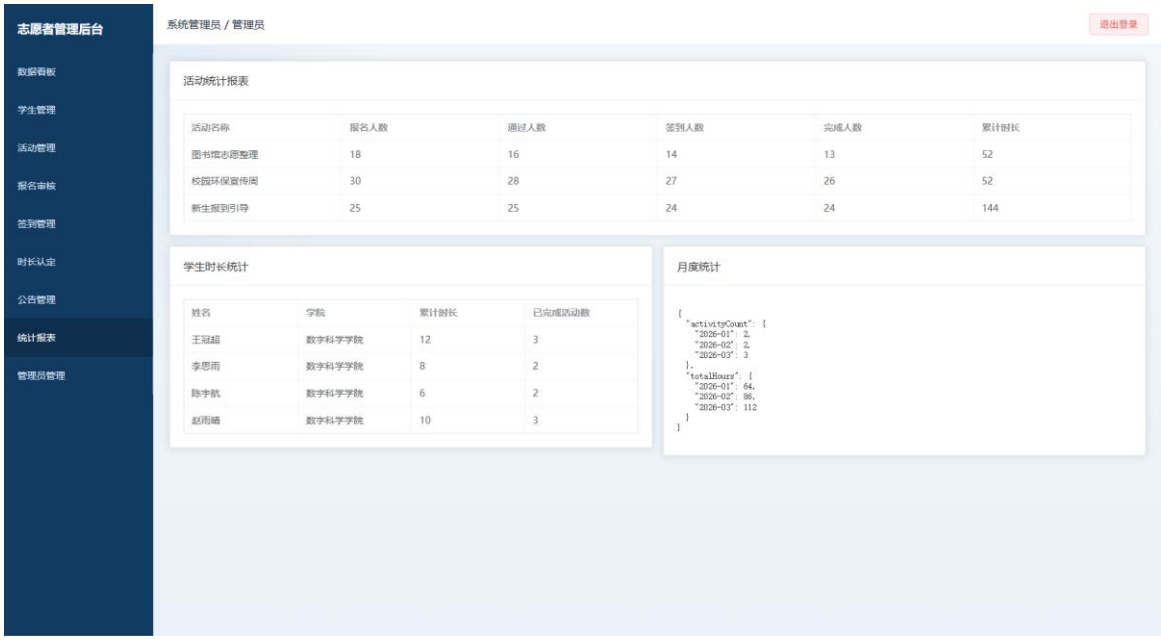

图 5-7 统计报表界面

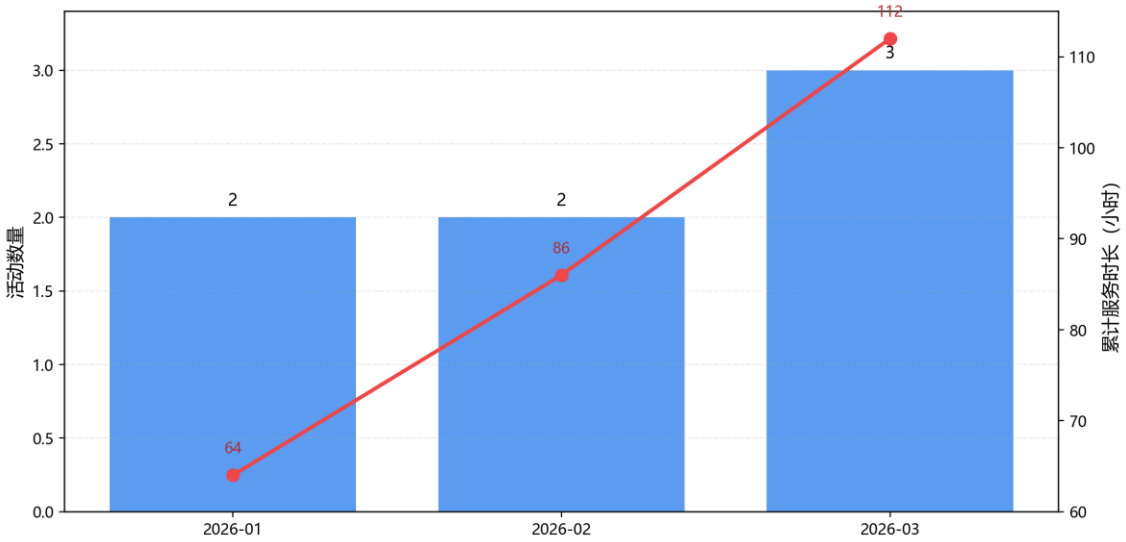

图 5-8 月度活动与服务时长统计示意图

6 系统测试与结果分析

6.1 测试环境与测试范围

为了验证系统功能闭环与运行稳定性，本文从开发构建验证、接口验证和页面展示验证三个层面开展测试。测试对象覆盖登录认证、活动管理、报名审核、签到签退、时长确认、公告消息和统计报表等关键功能，重点检验系统是否满足任务书提出的核心业务要求。

表 6-1 系统运行环境配置

项目	配置说明
操作系统	Windows 开发环境
后端运行环境	JDK 17 + Spring Boot 3.4.4
前端运行环境	Node.js + Vite 6.2.6
数据库	H2（开发/测试），保留 MySQL 扩展能力
主要验证方式	Maven 测试、前端构建、接口调用、页面截图检查

6.2 功能测试结果

根据项目当前实现情况，测试重点放在核心闭环功能。后端通过 `mvn test` 验证 Spring Boot 测试环境和 H2 数据初始化是否正常；前端通过 `npm run build` 验证工程化构建是否成功；接口层通过登录、管理员首页、学生首页等请求验证认证与聚合数据返回；页面层通过登录页、管理端和学生端截图核对界面结构与功能入口。整体上，系统核心功能已具备较完整的演示条件。

在测试策略上，本文并未局限于单一层面的“能打开页面”验证，而是将构建成功、接口返回正确、状态流转有效和界面展示完整四类结果结合起来分析。对于信息管理系统而言，真正重要的不仅是页面是否存在，更在于活动状态、报名状态、签到状态和时长状态之间能否保持一致。正因如此，测试章节才需要同时关注后端逻辑与前端呈现，而不能只做表面性的截图展示。

表 6-2 功能测试用例与结果

测试编号	测试内容	预期结果	实际结果
TC-01	管理员登录	返回 JWT 令牌与管理员资料	通过
TC-02	学生登录	返回 JWT 令牌与学生资料	通过

TC-03	活动发布	活动状态变更为已发布并生成签到码	通过
TC-04	报名审核通过	生成签到记录并推送审核通过消息	通过
TC-05	学生签到签退	状态按规则变更并生成时长记录	通过
TC-06	时长确认	学生累计服务时长正确更新	通过
TC-07	公告发布	学生消息列表收到公告提醒	通过
TC-08	统计报表查询	返回活动、时长等聚合数据	通过

6.3 结果分析

从测试结果看，系统在“活动发布 - 报名审核 - 签到签退 - 时长确认 - 消息通知 - 报表展示”这一核心链路上已经形成完整闭环。管理员能够通过后台集中处理学生、活动、时长和公告数据，学生则能够通过统一入口完成报名、签到和记录查询。对毕业设计项目而言，这表明系统不仅完成了基本页面堆叠，更在业务上形成了可解释、可验证的流程逻辑。

从实现质量看，项目在后端通过分层服务类集中承载业务规则，在数据层通过外键和唯一约束强化一致性，在前端则通过组件化页面保证交互统一。尤其是服务时长不直接手工累计、而由已确认明细汇总生成的设计，能够有效降低统计误差；活动发布时间与签到码自动生成机制，也使活动管理过程更加规范。这些设计都与论文前文的分析保持一致。

从示例数据的运行结果来看，系统已经能够围绕预置活动生成首页概览、类别分布和时长统计等图表信息，这说明数据库脚本、聚合逻辑和前端展示之间已建立有效联动。对于高校管理业务系统而言，能够在同一平台中将操作数据转化为分析结果，是系统从“记录工具”走向“管理工具”的重要标志。虽然当前数据量仍处于教学演示规模，但其结构和变化趋势已经足以支撑功能验证。

但测试同时也暴露出项目仍存在一些可优化之处。例如，当前密码存储机制适合教学场景但不宜直接用于高安全场景；前端部分页面在真实联调中仍有进一步增强异常提示和容错处理的空间；现阶段系统主要面向 PC 端浏览器，尚未针

对移动端或微信生态进行深度适配。因此，本文将该系统定位为“结构完整、功能闭环、便于扩展的校园志愿服务管理原型系统”。

此外，若将系统投入更复杂的真实环境，还需要进一步考虑并发报名控制、批量消息投递性能、文件存储治理以及跨学院协同管理等问题。这些问题在当前毕业设计阶段并未构成主要矛盾，但它们提示我们：一个管理系统从原型走向正式应用，不仅需要功能实现，更需要在安全、性能、运维和制度协同层面持续打磨。

6.4 本章小结

通过环境验证、功能测试和结果分析可以看出，本系统已经能够在本地环境下稳定完成大学生志愿服务管理的主要业务需求，具备答辩演示和后续优化迭代的基础。测试结果既说明了技术选型和架构设计的可行性，也为论文结论部分关于系统实用性的判断提供了直接依据。

7 结论与展望

7.1 研究结论

本文围绕高校志愿服务管理的现实需求，设计并实现了一套基于 Web 的大学生志愿者管理系统。通过对业务流程、角色职责、数据对象和技术体系的系统分析，本文完成了系统需求分析、数据库设计、总体架构设计、关键模块实现和测试验证。研究表明，采用 Vue 3 + Spring Boot + MyBatis-Plus 的技术路线，能够较好支撑活动发布、报名审核、签到签退、时长确认、公告消息和统计报表等核心功能，且系统结构清晰、演示完整、便于扩展。

从项目成果来看，本系统较好解决了传统人工管理中信息分散、效率较低、统计困难和记录不透明等问题。通过将报名、签到和时长认定串联为统一闭环，系统实现了对志愿活动全过程的数字化管理；通过图表与报表展示，管理者能够快速掌握活动执行情况和学生参与情况；通过个人消息和时长查询，学生也能够及时获得反馈。该系统符合高校志愿服务信息化建设的基本方向。

7.2 后续展望

受毕业设计时间与项目边界限制，系统仍有进一步优化空间。首先，在安全层面，可将当前密码摘要方案升级为更强的口令哈希算法，并增加细粒度权限控制、异常登录监测和更完善的审计能力；其次，在交互层面，可进一步优化移动端适配、扫码签到、消息订阅和操作提示机制，提高系统在实际校园环境中的使用体验；再次，在数据分析层面，可引入更丰富的志愿画像、学院维度比较和活动效果评价指标，为志愿服务治理提供更深入的数据支撑。

总体而言，本文完成的系统已经具备较好的教学展示价值和一定的应用推广意义。未来若结合真实校园业务数据和长期运行反馈持续迭代，该系统有望从毕业设计项目进一步演化为更完整的校园志愿服务管理平台。

参考文献

[1] Fielding R T. Architectural Styles and the Design of Network-based Software Architectures[D/OL]. Irvine: University of California, 2000[2026-03-20]. Available: https://ics.uci.edu/~fielding/pubs/dissertation/top.htm.

[2] Oracle. JDK 17 Documentation[EB/OL]. [2026-03-20]. https://docs.oracle.com/en/java/javase/17/.

[3] Spring Team. Spring Boot Reference Documentation[EB/OL]. [2026-03-20]. https://docs.spring.io/spring-boot/reference/.

[4] Vue.js. Vue.js Guide[EB/OL]. [2026-03-20]. https://vuejs.org/guide/introduction.html.

[5] Vue Router Team. Vue Router Guide[EB/OL]. [2026-03-20]. https://router.vuejs.org/guide/.

[6] Element Plus Contributors. Element Plus Documentation[EB/OL]. [2026-03-20]. https://element-plus.org/en-US/guide/installation.html.

[7] Apache Software Foundation. Apache ECharts Handbook[EB/OL]. [2026-03-20]. https://echarts.apache.org/handbook/en/get-started/.

[8] baomidou. MyBatis-Plus Getting Started[EB/OL]. [2026-03-20]. https://baomidou.com/en/getting-started/.

[9] Jones M, Bradley J, Sakimura N. JSON Web Token (JWT): RFC 7519[EB/OL]. [2026-03-20]. https://www.rfc-editor.org/rfc/rfc7519.html.

[10] H2 Group. H2 Database Engine Features[EB/OL]. [2026-03-20]. https://h2database.github.io/html/features.html.

[11] Oracle. MySQL Reference Manual[EB/OL]. [2026-03-20]. https://dev.mysql.com/doc/refman/8.4/en/.

[12] VoidZero. Vite Guide[EB/OL]. [2026-03-20]. https://vite.dev/guide/.

[13] 中华人民共和国国务院. 中华人民共和国国务院令（第 685 号） 志愿服务条例[EB/OL]. [2026-03-20]. https://www.gov.cn/gongbao/content/2017/content_5225860.htm.

[14] 中央社会工作部. 中央社会工作部有关负责人就《关于健全新时代志愿服务体系的意见》答记者问[EB/OL]. [2026-03-20]. https://www.gov.cn/zhengce/202404/content_6946910.htm.

致　谢

在本论文撰写及系统开发过程中，我得到了许多老师和同学的帮助与支持。在此，谨向所有给予我帮助的师长和同伴表示诚挚感谢。

首先，衷心感谢指导教师朱媛媛老师。在课题选题、研究思路梳理、系统设计分析以及论文结构调整等方面，老师始终给予我耐心细致的指导。老师严谨认真的治学态度和求真务实的工作作风，使我深刻体会到毕业论文不仅是一次技术训练，更是一次规范科研写作的系统实践。

其次，感谢项目开发和测试过程中给予我建议的同学与朋友。无论是在界面交互细节、功能联调还是论文修改阶段，大家都提出了许多具有参考价值的意见，使本系统得以不断完善。

最后，感谢家人一直以来给予我的理解与支持。正是因为他们在学习和生活中的鼓励，我才能够较为顺利地完成毕业设计任务。未来我将继续保持严谨、踏实的学习态度，在后续学习和工作中不断提升自身专业能力。